

```
//UDP length field
iph->tot_len = htons(total_len - sizeof(struct ethhdr));
//IP length field
```

The IP header checksum

There is one more field remaining in the IP header check, which is used to have a checksum. A checksum is used for error checking of the header.

When the packet arrives at the router, it calculates the checksum, and if the calculated checksum does not match with the checksum field of the header, the router will drop the packet; and if it matches, the router will decrement the time to the live field by one, and forward it.

To calculate the checksum, sum up all the 16-bit words of the IP header and if there is any carry, add it again to get a 16-bit word. After this, find the complement of 1's and that is our checksum. To check whether our checksum is correct, use the above algorithm.

```
unsigned short checksum(unsigned short* buff, int _16bitword)
{
    unsigned long sum;
    for(sum=0; _16bitword>0; _16bitword--)
        sum+=htons(*(buff)++);
    sum = ((sum >> 16) + (sum & 0xFFFF));
```

```
sum += (sum>>16);
return (unsigned short)(~sum);
}
```

```
iph->check = checksum((unsigned short*)(sendbuff +
sizeof(struct ethhdr)), (sizeof(struct iphdr)/2));
```

Sending the packet

Now we have our packet but before sending it, let's fill the *socaddr_ll* structure with the destination MAC address:

```
struct sockaddr_ll saddr_ll;
saddr_ll.sll_ifindex = ifreq_i.ifr_ifindex; // index of
interface
saddr_ll.sll_halen = ETH_ALEN; // length of destination mac
address
saddr_ll.sll_addr[0] = DESTMAC0;
saddr_ll.sll_addr[1] = DESTMAC1;
saddr_ll.sll_addr[2] = DESTMAC2;
saddr_ll.sll_addr[3] = DESTMAC3;
saddr_ll.sll_addr[4] = DESTMAC4;
saddr_ll.sll_addr[5] = DESTMAC5;
```

And now it's time to send it, for which let's use the *sendto* API:

```
send_len = sendto(sock_raw, sendbuff, 64, 0, (const struct
sockaddr*)&saddr_ll, sizeof(struct sockaddr_ll));
if(send_len<0)
{
    printf("error in sending....sendlen=%d....errno=%d\n", send_
len, errno);
    return -1;
}
```

How to run the program

Go to *root* user, then compile and run your program in a machine. And in another machine, or in your destination machine, run the packet sniffer program as the *root* user and analyse the data that you are sending.

What to do next

We made a packet sniffer as well as a packet sender, but this is a user space task. Now let's try the same things in kernel space. For this, try to understand *struct sk_buff* and make a module that can perform the same things in kernel space.

You can get the complete code used in this article from <http://1drv.ms/14yN6jr> 

By: Subodh Saxena

The author works at Effortsys Technologies in the network device driver space. He is interested in new technologies, and especially open source technologies.



2 Simple Ways To Stay Up-to-date With India's Electronics Industry

Want Latest News?
Check out
EFYTimes.com



Want Industry Trends?
Check out
Electronicsb2b.com



For any queries, please contact our team at efyeng@efyindia.com OR +91-11-26810601